MANS360 Database Schema Extension

Sprint 9: Doctor Partner & QR Referral System

🔍 CURRENT STATE ANALYSIS

Existing Tables (from Architecture docs):

```sql
-- EXISTING: Core user management
users (id, email, phone, role, subscription_tier, ...)
therapist_profiles (user_id, nmc_registration, specializations, ...)
patients (patient_id, name, email, phone, ...)
assessments (assessment_id, patient_id, phq9_score, gad7_score, ...)
leads (lead_id, assessment_id, match_score, lead_tier, ...)
sessions (session_id, patient_id, therapist_id, ...)

-- EXISTING: Analytics (Data Warehouse)
fact_qr_tracking (for marketing QR campaigns)
dim_campaigns
```

Gap Identified:

❌ **No `doctor_profiles` table** — Referring doctors (GPs, specialists) are different from platform therapists ❌ **No `doctor_referrals` table** — Need to track patient referrals from doctors ❌ **No `doctor_credits` table** — Incentive system for referring doctors ❌ **QR tracking exists but not doctor-specific** — Need doctor attribution

---

✅ PROPOSED SCHEMA ADDITIONS

Option A: Extend Existing `users` Table (Recommended)

Add `role = 'referring_doctor'` to existing users table, with a new `doctor_profiles` table.

```sql
-- ============================================================
-- OPTION A: EXTEND EXISTING SCHEMA (RECOMMENDED)
-- ============================================================

-- 1. Add new role to users table constraint
-- (Modify existing CHECK constraint)
ALTER TABLE users
DROP CONSTRAINT users_role_check;

ALTER TABLE users
ADD CONSTRAINT users_role_check
CHECK (role IN ('patient', 'therapist', 'admin', 'referring_doctor', 'coach'));

-- 2. Create doctor_profiles table (for referring doctors, NOT therapists)
CREATE TABLE doctor_profiles (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,

    -- Professional info
    doctor_code VARCHAR(20) UNIQUE NOT NULL,  -- DR-KA-0042
    full_name VARCHAR(255) NOT NULL,
    credentials VARCHAR(255),                  -- MBBS, MD, etc.
    registration_number VARCHAR(50),           -- MCI/State council number
    registration_council VARCHAR(100),         -- Karnataka Medical Council
    specialization VARCHAR(100),               -- General Physician,
Cardiologist
    hospital_clinic VARCHAR(255),

    -- Location
    city VARCHAR(100),
    state VARCHAR(50),
    pincode VARCHAR(10),

    -- Contact (may differ from user table)
    clinic_phone VARCHAR(20),
    clinic_email VARCHAR(255),

    -- Experience
    experience_years INT,
    photo_url VARCHAR(500),

    -- Verification
    verification_status VARCHAR(20) DEFAULT 'pending'
        CHECK (verification_status IN ('pending', 'verified', 'rejected',
'suspended')),
    verified_at TIMESTAMP,
    verified_by UUID REFERENCES users(id),
```

```
    rejection_reason TEXT,

    -- Documents (stored in S3)
    registration_doc_url VARCHAR(500),
    id_proof_url VARCHAR(500),

    -- QR & Referral
    referral_url VARCHAR(255),                -- https://mans360.com/r/DR-KA-0042
    qr_code_url VARCHAR(500),                 -- https://cdn.mans360.ai/qr/dr-ka-0042.png
    qr_generated_at TIMESTAMP,

    -- Preferences
    notify_on_referral BOOLEAN DEFAULT TRUE,
    notification_method VARCHAR(20) DEFAULT 'whatsapp'
        CHECK (notification_method IN ('email', 'whatsapp', 'sms', 'app')),

    -- Tier (for incentive multiplier)
    referral_tier VARCHAR(20) DEFAULT 'bronze'
        CHECK (referral_tier IN ('bronze', 'silver', 'gold', 'platinum')),

    -- Timestamps
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW(),

    -- Constraints
    UNIQUE(user_id),
    UNIQUE(registration_number, registration_council)
);

-- Index for fast lookup
CREATE INDEX idx_doctor_profiles_code ON doctor_profiles(doctor_code);
CREATE INDEX idx_doctor_profiles_state ON doctor_profiles(state);
CREATE INDEX idx_doctor_profiles_verification ON doctor_profiles(verification_status);


-- 3. Doctor referral tracking table
CREATE TABLE doctor_referrals (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

    -- Doctor who referred
    doctor_id UUID REFERENCES doctor_profiles(id) ON DELETE SET NULL,
    doctor_code VARCHAR(20),                  -- Denormalized for faster queries
```

```
    -- Session tracking
    session_id VARCHAR(100) NOT NULL,          -- Unique session for
attribution

    -- Scan event
    scan_timestamp TIMESTAMP DEFAULT NOW(),
    device_type VARCHAR(50),                   -- mobile, tablet, desktop
    device_os VARCHAR(50),                     -- iOS, Android, Windows
    browser VARCHAR(50),
    ip_address VARCHAR(50),
    city VARCHAR(100),
    state VARCHAR(50),
    country VARCHAR(10) DEFAULT 'IN',
    user_agent TEXT,

    -- Funnel progression
    landing_page_viewed_at TIMESTAMP,
    signup_started_at TIMESTAMP,
    signup_completed_at TIMESTAMP,
    assessment_completed_at TIMESTAMP,
    subscription_started_at TIMESTAMP,

    -- Patient (after signup)
    patient_id UUID REFERENCES patients(patient_id) ON DELETE SET NULL,

    -- Status
    status VARCHAR(30) DEFAULT 'scanned'
        CHECK (status IN ('scanned', 'landed', 'signup_started', 'signed_up',
                          'assessment_done', 'subscribed', 'churned',
'expired')),

    -- Subscription details (if converted)
    subscription_tier VARCHAR(20),             -- free, tier2, tier3
    subscription_amount DECIMAL(10,2),
    subscription_start_date DATE,

    -- Lifetime value tracking
    lifetime_value DECIMAL(10,2) DEFAULT 0,
    months_active INT DEFAULT 0,

    -- Attribution window
    attribution_expires_at TIMESTAMP,          -- 30 days from scan

    -- Timestamps
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);
```

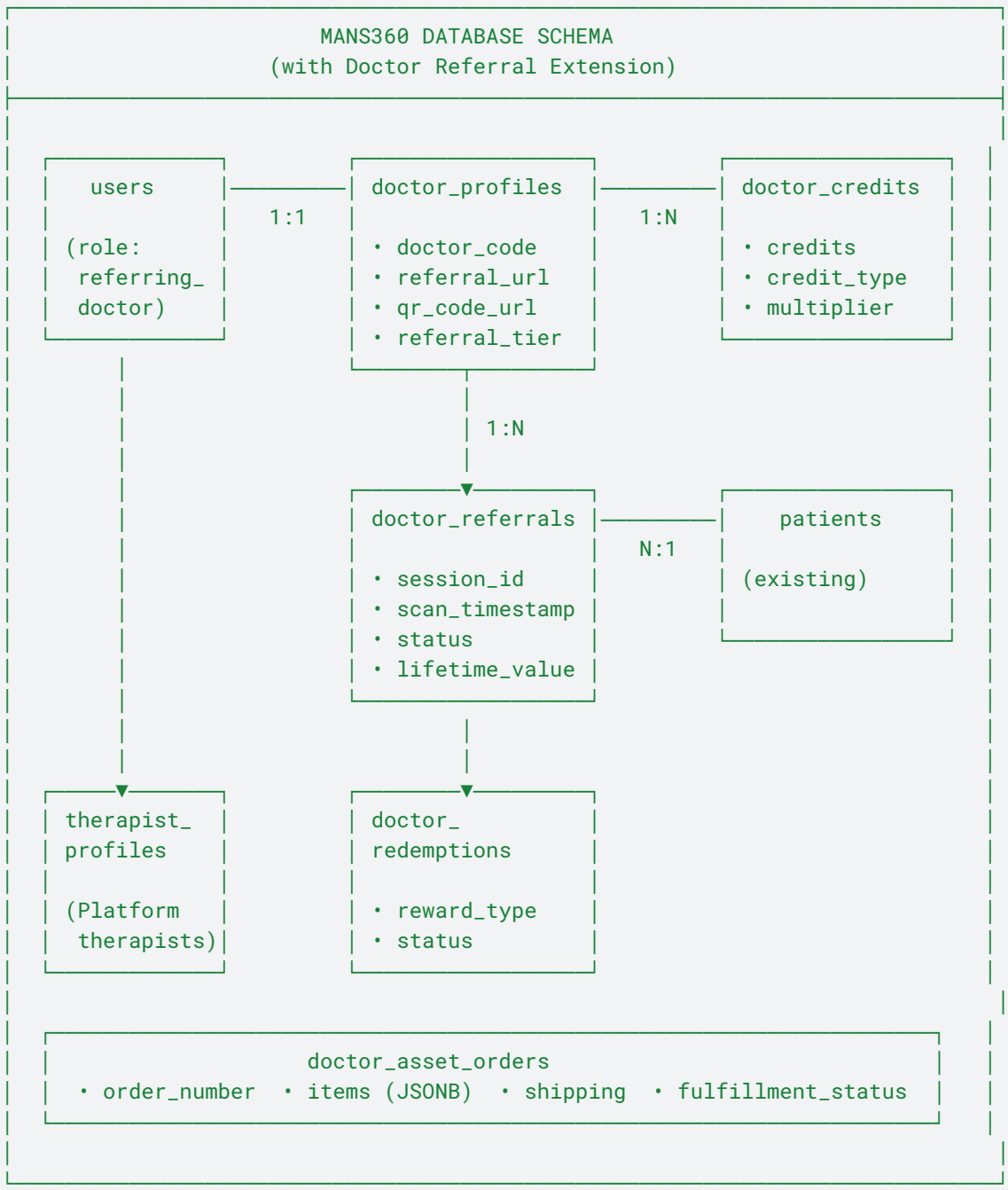
```sql
-- Indexes for analytics
CREATE INDEX idx_doctor_referrals_doctor ON doctor_referrals(doctor_id);
CREATE INDEX idx_doctor_referrals_session ON doctor_referrals(session_id);
CREATE INDEX idx_doctor_referrals_status ON doctor_referrals(status);
CREATE INDEX idx_doctor_referrals_scan_date ON
doctor_referrals(scan_timestamp);
CREATE INDEX idx_doctor_referrals_patient ON doctor_referrals(patient_id);


-- 4. Doctor credits/rewards tracking
CREATE TABLE doctor_credits (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

    -- Doctor
    doctor_id UUID REFERENCES doctor_profiles(id) ON DELETE CASCADE,

    -- Related referral (if applicable)
    referral_id UUID REFERENCES doctor_referrals(id) ON DELETE SET NULL,

    -- Credit details
    credits INT NOT NULL,
    credit_type VARCHAR(30) NOT NULL
        CHECK (credit_type IN ('signup', 'assessment', 'subscription_tier2',
                               'subscription_tier3', 'retention_bonus',
                               'manual_bonus', 'manual_deduction',
'redemption')),

    -- Multiplier applied (based on tier)
    base_credits INT,
    multiplier DECIMAL(3,2) DEFAULT 1.0,

    -- Description
    description VARCHAR(255),

    -- Admin tracking (for manual adjustments)
    created_by UUID REFERENCES users(id),

    -- Timestamps
    created_at TIMESTAMP DEFAULT NOW()
);

-- Index for balance calculation
CREATE INDEX idx_doctor_credits_doctor ON doctor_credits(doctor_id);
CREATE INDEX idx_doctor_credits_type ON doctor_credits(credit_type);
```

```sql
-- 5. Credit redemption requests
CREATE TABLE doctor_redemptions (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

    -- Doctor
    doctor_id UUID REFERENCES doctor_profiles(id) ON DELETE CASCADE,

    -- Redemption details
    credits_redeemed INT NOT NULL,
    reward_type VARCHAR(50) NOT NULL,          -- amazon_voucher,
bank_transfer, premium_subscription, etc.
    reward_value VARCHAR(255),                 -- ₹500, 1 year premium, etc.

    -- Bank details (for bank transfers)
    bank_account_number VARCHAR(20),
    bank_ifsc VARCHAR(15),
    bank_name VARCHAR(100),
    account_holder_name VARCHAR(255),

    -- Processing
    status VARCHAR(20) DEFAULT 'pending'
        CHECK (status IN ('pending', 'processing', 'completed', 'failed',
'cancelled')),
    processed_at TIMESTAMP,
    processed_by UUID REFERENCES users(id),
    failure_reason TEXT,

    -- Reference (voucher code, transaction ID, etc.)
    external_reference VARCHAR(255),

    -- Timestamps
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- Index
CREATE INDEX idx_doctor_redemptions_doctor ON doctor_redemptions(doctor_id);
CREATE INDEX idx_doctor_redemptions_status ON doctor_redemptions(status);


-- 6. Physical asset orders (desk cards, posters)
CREATE TABLE doctor_asset_orders (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

    -- Doctor
    doctor_id UUID REFERENCES doctor_profiles(id) ON DELETE CASCADE,
```

```
    -- Order details
    order_number VARCHAR(20) UNIQUE,

    -- Items
    items JSONB NOT NULL,                       -- [{"type": "desk_card", "qty":
2}, ...]
    total_amount DECIMAL(10,2),

    -- Shipping
    shipping_name VARCHAR(255),
    shipping_address TEXT,
    shipping_city VARCHAR(100),
    shipping_state VARCHAR(50),
    shipping_pincode VARCHAR(10),
    shipping_phone VARCHAR(20),

    -- Payment
    payment_status VARCHAR(20) DEFAULT 'pending'
        CHECK (payment_status IN ('pending', 'paid', 'failed', 'refunded')),
    payment_id VARCHAR(100),                    -- Razorpay payment ID
    paid_at TIMESTAMP,

    -- Fulfillment
    fulfillment_status VARCHAR(20) DEFAULT 'pending'
        CHECK (fulfillment_status IN ('pending', 'processing', 'shipped',
'delivered', 'cancelled')),
    shipped_at TIMESTAMP,
    tracking_number VARCHAR(100),
    tracking_url VARCHAR(500),
    delivered_at TIMESTAMP,

    -- Timestamps
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- Index
CREATE INDEX idx_doctor_asset_orders_doctor ON doctor_asset_orders(doctor_id);
CREATE INDEX idx_doctor_asset_orders_status ON
doctor_asset_orders(fulfillment_status);
```

🔗 RELATIONSHIP DIAGRAM

None



📊 KEY VIEWS & FUNCTIONS

```sql
-- ============================================================
-- USEFUL VIEWS
-- ============================================================

-- 1. Doctor dashboard stats view
CREATE VIEW v_doctor_dashboard AS
SELECT
    dp.id AS doctor_id,
    dp.doctor_code,
    dp.full_name,
    dp.referral_tier,

    -- Referral counts
    COUNT(dr.id) AS total_referrals,
    COUNT(CASE WHEN dr.scan_timestamp >= DATE_TRUNC('month', NOW()) THEN 1 END)
AS this_month_referrals,
    COUNT(CASE WHEN dr.status = 'signed_up' OR dr.status IN ('assessment_done',
'subscribed') THEN 1 END) AS total_signups,
    COUNT(CASE WHEN dr.status = 'subscribed' THEN 1 END) AS
total_subscriptions,

    -- Conversion rate
    ROUND(
        COUNT(CASE WHEN dr.status IN ('signed_up', 'assessment_done',
'subscribed') THEN 1 END)::DECIMAL /
        NULLIF(COUNT(dr.id), 0) * 100, 2
    ) AS conversion_rate,

    -- Revenue
    COALESCE(SUM(dr.lifetime_value), 0) AS total_patient_value,

    -- Credits
    COALESCE(
        (SELECT SUM(credits) FROM doctor_credits WHERE doctor_id = dp.id), 0
    ) AS total_credits_earned,
    COALESCE(
        (SELECT SUM(credits_redeemed) FROM doctor_redemptions WHERE doctor_id =
dp.id AND status = 'completed'), 0
    ) AS total_credits_redeemed

FROM doctor_profiles dp
LEFT JOIN doctor_referrals dr ON dp.id = dr.doctor_id
GROUP BY dp.id, dp.doctor_code, dp.full_name, dp.referral_tier;
```

```sql
-- 2. Doctor credit balance view
CREATE VIEW v_doctor_credit_balance AS
SELECT
    dp.id AS doctor_id,
    dp.doctor_code,
    dp.full_name,
    COALESCE(SUM(dc.credits), 0) AS credits_earned,
    COALESCE(
        (SELECT SUM(credits_redeemed) FROM doctor_redemptions
         WHERE doctor_id = dp.id AND status IN ('completed', 'processing')), 0
    ) AS credits_used,
    COALESCE(SUM(dc.credits), 0) - COALESCE(
        (SELECT SUM(credits_redeemed) FROM doctor_redemptions
         WHERE doctor_id = dp.id AND status IN ('completed', 'processing')), 0
    ) AS credits_available
FROM doctor_profiles dp
LEFT JOIN doctor_credits dc ON dp.id = dc.doctor_id
GROUP BY dp.id, dp.doctor_code, dp.full_name;


-- ============================================================
-- USEFUL FUNCTIONS
-- ============================================================

-- 1. Generate unique doctor code
CREATE OR REPLACE FUNCTION generate_doctor_code(p_state VARCHAR)
RETURNS VARCHAR AS $$
DECLARE
    state_code VARCHAR(2);
    next_num INT;
    new_code VARCHAR(20);
BEGIN
    -- Map state to 2-letter code
    state_code := CASE p_state
        WHEN 'Karnataka' THEN 'KA'
        WHEN 'Maharashtra' THEN 'MH'
        WHEN 'Delhi' THEN 'DL'
        WHEN 'Tamil Nadu' THEN 'TN'
        WHEN 'Telangana' THEN 'TS'
        WHEN 'Kerala' THEN 'KL'
        WHEN 'Gujarat' THEN 'GJ'
        WHEN 'West Bengal' THEN 'WB'
        WHEN 'Uttar Pradesh' THEN 'UP'
        WHEN 'Rajasthan' THEN 'RJ'
        ELSE 'XX'
```

```
    END;

    -- Get next number for this state
    SELECT COALESCE(MAX(
        CAST(SUBSTRING(doctor_code FROM 'DR-' || state_code || '-(\d+)') AS
INT)
    ), 0) + 1
    INTO next_num
    FROM doctor_profiles
    WHERE doctor_code LIKE 'DR-' || state_code || '-%';

    -- Format: DR-KA-0042
    new_code := 'DR-' || state_code || '-' || LPAD(next_num::TEXT, 4, '0');

    RETURN new_code;
END;
$$ LANGUAGE plpgsql;


-- 2. Award credits to doctor
CREATE OR REPLACE FUNCTION award_doctor_credits(
    p_doctor_id UUID,
    p_referral_id UUID,
    p_credit_type VARCHAR,
    p_base_credits INT,
    p_description VARCHAR DEFAULT NULL
) RETURNS INT AS $$
DECLARE
    v_multiplier DECIMAL(3,2);
    v_final_credits INT;
BEGIN
    -- Get doctor's tier multiplier
    SELECT CASE referral_tier
        WHEN 'bronze' THEN 1.0
        WHEN 'silver' THEN 1.25
        WHEN 'gold' THEN 1.5
        WHEN 'platinum' THEN 2.0
        ELSE 1.0
    END INTO v_multiplier
    FROM doctor_profiles
    WHERE id = p_doctor_id;

    -- Calculate final credits
    v_final_credits := ROUND(p_base_credits * v_multiplier);

    -- Insert credit record
    INSERT INTO doctor_credits (
```

```
        doctor_id, referral_id, credits, credit_type,
        base_credits, multiplier, description
    ) VALUES (
        p_doctor_id, p_referral_id, v_final_credits, p_credit_type,
        p_base_credits, v_multiplier, p_description
    );

    -- Check for tier upgrade
    PERFORM check_doctor_tier_upgrade(p_doctor_id);

    RETURN v_final_credits;
END;
$$ LANGUAGE plpgsql;


-- 3. Check and upgrade doctor tier
CREATE OR REPLACE FUNCTION check_doctor_tier_upgrade(p_doctor_id UUID)
RETURNS VOID AS $$
DECLARE
    v_total_referrals INT;
    v_current_tier VARCHAR(20);
    v_new_tier VARCHAR(20);
BEGIN
    -- Count total successful referrals
    SELECT COUNT(*) INTO v_total_referrals
    FROM doctor_referrals
    WHERE doctor_id = p_doctor_id
    AND status IN ('signed_up', 'assessment_done', 'subscribed');

    -- Get current tier
    SELECT referral_tier INTO v_current_tier
    FROM doctor_profiles
    WHERE id = p_doctor_id;

    -- Determine new tier
    v_new_tier := CASE
        WHEN v_total_referrals >= 100 THEN 'platinum'
        WHEN v_total_referrals >= 50 THEN 'gold'
        WHEN v_total_referrals >= 25 THEN 'silver'
        ELSE 'bronze'
    END;

    -- Update if changed
    IF v_new_tier != v_current_tier THEN
        UPDATE doctor_profiles
        SET referral_tier = v_new_tier, updated_at = NOW()
        WHERE id = p_doctor_id;
```

```
        -- TODO: Trigger notification for tier upgrade
    END IF;
END;
$$ LANGUAGE plpgsql;


-- ============================================================
-- TRIGGERS
-- ============================================================

-- 1. Auto-update updated_at timestamp
CREATE OR REPLACE FUNCTION update_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tr_doctor_profiles_updated
    BEFORE UPDATE ON doctor_profiles
    FOR EACH ROW EXECUTE FUNCTION update_updated_at();

CREATE TRIGGER tr_doctor_referrals_updated
    BEFORE UPDATE ON doctor_referrals
    FOR EACH ROW EXECUTE FUNCTION update_updated_at();

CREATE TRIGGER tr_doctor_redemptions_updated
    BEFORE UPDATE ON doctor_redemptions
    FOR EACH ROW EXECUTE FUNCTION update_updated_at();


-- 2. Auto-generate referral URL when doctor verified
CREATE OR REPLACE FUNCTION generate_referral_url()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.verification_status = 'verified' AND OLD.verification_status !=
'verified' THEN
        NEW.referral_url := 'https://mans360.com/r/' || NEW.doctor_code;
        NEW.qr_generated_at := NOW();
        -- QR code URL generated by application layer
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```sql
CREATE TRIGGER tr_doctor_verified
    BEFORE UPDATE ON doctor_profiles
    FOR EACH ROW EXECUTE FUNCTION generate_referral_url();
```

🔒 ROW-LEVEL SECURITY (DPDPA Compliance)

```sql
-- Enable RLS
ALTER TABLE doctor_profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE doctor_referrals ENABLE ROW LEVEL SECURITY;
ALTER TABLE doctor_credits ENABLE ROW LEVEL SECURITY;
ALTER TABLE doctor_redemptions ENABLE ROW LEVEL SECURITY;

-- Doctors can only see their own profile
CREATE POLICY "Doctors see own profile" ON doctor_profiles
    FOR SELECT USING (user_id = auth.uid());

-- Doctors can update their own profile (except verification fields)
CREATE POLICY "Doctors update own profile" ON doctor_profiles
    FOR UPDATE USING (user_id = auth.uid());

-- Doctors can only see their own referrals
CREATE POLICY "Doctors see own referrals" ON doctor_referrals
    FOR SELECT USING (
        doctor_id IN (SELECT id FROM doctor_profiles WHERE user_id =
auth.uid())
    );

-- Doctors can only see their own credits
CREATE POLICY "Doctors see own credits" ON doctor_credits
    FOR SELECT USING (
        doctor_id IN (SELECT id FROM doctor_profiles WHERE user_id =
auth.uid())
    );

-- Admins can see everything
CREATE POLICY "Admins see all" ON doctor_profiles
    FOR ALL USING (
        EXISTS (SELECT 1 FROM users WHERE id = auth.uid() AND role = 'admin')
    );
```

📈 INTEGRATION WITH EXISTING TABLES

```sql
-- Add referral tracking to patients table
ALTER TABLE patients
ADD COLUMN referred_by_doctor UUID REFERENCES doctor_profiles(id),
ADD COLUMN referral_session_id VARCHAR(100);

-- Add referral tracking to assessments table
ALTER TABLE assessments
ADD COLUMN referral_id UUID REFERENCES doctor_referrals(id);

-- Add to fact_qr_tracking (data warehouse)
-- This already exists, just add doctor-specific dimensions
ALTER TABLE fact_qr_tracking
ADD COLUMN doctor_id UUID,
ADD COLUMN doctor_code VARCHAR(20),
ADD COLUMN is_doctor_referral BOOLEAN DEFAULT FALSE;
```

✅ SUMMARY: SCHEMA CHANGES NEEDED

Table	Action	Purpose
`users`	Modify constraint	Add 'referring_doctor' role
`doctor_profiles`	**NEW**	Store referring doctor profiles
`doctor_referrals`	**NEW**	Track patient referrals from doctors
`doctor_credits`	**NEW**	Incentive/rewards tracking
`doctor_redemptions`	**NEW**	Credit redemption requests
`doctor_asset_orders`	**NEW**	Physical asset (cards, posters) orders
`patients`	Add columns	`referred_by_doctor`, `referral_session_id`

Table	Action	Purpose
`assessments`	Add column	`referral_id`
`fact_qr_tracking`	Add columns	Doctor attribution in analytics

🚀 MIGRATION SCRIPT

```sql
-- Run this in order during Sprint 9 deployment

BEGIN;

-- 1. Extend users role constraint
ALTER TABLE users DROP CONSTRAINT IF EXISTS users_role_check;
ALTER TABLE users ADD CONSTRAINT users_role_check
    CHECK (role IN ('patient', 'therapist', 'admin', 'referring_doctor',
'coach'));

-- 2. Create new tables (in order due to foreign keys)
-- [Run CREATE TABLE statements from above]

-- 3. Create views
-- [Run CREATE VIEW statements from above]

-- 4. Create functions
-- [Run CREATE FUNCTION statements from above]

-- 5. Create triggers
-- [Run CREATE TRIGGER statements from above]

-- 6. Enable RLS
-- [Run ALTER TABLE ... ENABLE ROW LEVEL SECURITY statements]

-- 7. Add columns to existing tables
ALTER TABLE patients
ADD COLUMN IF NOT EXISTS referred_by_doctor UUID,
ADD COLUMN IF NOT EXISTS referral_session_id VARCHAR(100);

COMMIT;
```

Document Version: 1.0
Sprint: 9
Compatible with: MANS360-Refined-Architecture-Complete.md
Database: PostgreSQL 14+